

Backend проекта: MCP для структуры, SQL для данных

Как через MCP узнали таблицы хранения, подключились к SQL Server и вывели показатели в визуал — без зависимости дашборда от MCP

- MCP — только чтобы узнать структуру SQL (имена таблиц и полей 1С)
- Дашборд ходит в SQL Server напрямую: запросы не зависят от MCP
- Базы: msk_uat_copy3 (УАТ) + msk_buh_copy (БП) на 192.168.80.10

Архитектура backend

Два контура: MCP для разведки, SQL для дашборда

КОНТУР А — разведка (разово / по необходимости)



нет зависимости в рантайме: SQL-запросы дашборда не ходят в MCP и не требуют его

КОНТУР В — дашборд в рантайме (без MCP)



Шаг 1. MCP → структура SQL

MCP использовали, чтобы узнать, как 1С хранит данные в СУБД

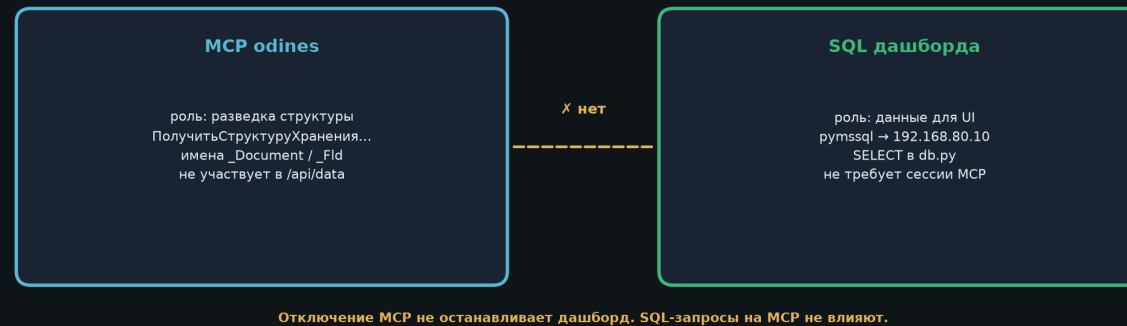


MCP нужен только чтобы узнать структуру SQL. Дальше дашборд ходит в СУБД сам.

- Транспорт HTTP: `http://192.168.80.15:6003/mcp` (Authorization: Bearer ...)
- Инструменты: `execute_code`, `get_metadata` (`extension_name` = «Расширение1»)
- Ключ: `ПолучитьСтруктуруХраненияБазыДанных`(, Истина) → `_Document...` / `_Fld...`
- Пример: `Документ.ак_Смены` → `_Document18335`, `ТЧ Товары` → `_Document18335_VT18350`
- После маппинга MCP для работы дашборда не нужен — имена уже в `db.py`

МСР и SQL независимы

Запросы дашборда не влияют на МСР и не зависят от его подключения



- МСР — инструмент разработки/разведки: один раз узнали структуру хранения в SQL
- Дашборд (Flask + pymssql) подключается к 192.168.80.10 сам, через .env
- SELECT в db.py и /api/data не вызывают МСР и не держат сессию 1С
- Можно выключить МСР — КРІ и графики продолжают работать, пока доступен SQL
- Обратно: нагрузка SQL дашборда не затрагивает МСР-сервер odines

Шаг 2. Подключение к SQL Server

Прямой доступ дашборда — без MCP

Разделение ролей хостов

- 192.168.80.15 — сервер приложений 1С
- Порты 1433/1434 на нём закрыты
- СУБД живёт на 192.168.80.10
- УАТ: msk_uat_copy3
- БП: msk_buh_copy
- Логин: user1c (пароль в .env / DPAPI)

Клиенты

- Python: pymssql (UTF-8)
- без вызовов MCP odines
- PowerShell: tools/sql-1c.ps1
- Часто нужен -ReadWriteIntent
- Flask читает SQL_PASSWORD
- из dashboard/.env

Фрагмент подключения (db.py) — только SQL, MCP не участвует:

```
pymssql.connect(server='192.168.80.10', user='user1c', password=..., database='msk_uat_copy3', charset='utf8')
```

Даты 1С в SQL со смещением +2000 лет → везде DATEADD(year, -2000, _Date_Time)

Шаг 3. Схема хранения

Объекты 1С → таблицы SQL

УАТ msk_uat_copy3

БП msk_buh_copy

_Document18335 ак_Смены

_Date_Time (+2000 лет)
_Fld18344RRef бригада
_Fld18341RRef цех
_Marked удаление
_IDRRef

_Document18335_VT18350 Товары

_Document18335_IDRRef
_Fld18354RRef номенклатура
_Fld18353 количество
_LineNo18351

имя

_Document637 РеализацияТоваровУслуг

_Posted = 0x01 проведён
ТЧ: qty / price / amount
_Reference204 номенклатура
_Reference177 контрагент
join УАТ+БП по очищенному имени

_Document18337 ак_Отгрузка

_Fld18387 вес тюков
_Fld18389 вес весовой
усушка = тюки - весовая

_Document18336 ак_Тюк

статус EnumOrder=1
«На складе»
остаток на сегодня

Справочники УАТ

_Reference106X1 бригады
_Reference90 подразделения
_Reference82 номенклатура / фракции

Шаг 4. Прямые SQL-запросы

Что считает backend в db.py — без обращения к MCP

SHIFTS_QUERY

ак_Смены + ТЧ Товары · выработка, бригады, цеха, фракции

SHIPMENTS_QUERY

ак_Отгрузка · вывезено по весовой, усушка по рейсам

WAREHOUSE_QUERY

ак_Тюк + статус · остаток склада «на сегодня»

SALES_PERIOD_QUERY

Реализация (БП) · выручка, тонны, покупатели

SALES_12M_QUERY

Реализация 12 мес. · индекс цен по фракциям

Шаг 5. Python → API → визуал

Flask отдаёт JSON, фронт рисует KPI и блоки



API

- GET / — HTML дашборд
- GET /api/data?from=&to= — JSON
- build_dashboard(from, to) агрегирует
- границу «по» делает exclusive (+1 день)
- хост: 127.0.0.1:5050

UI-блоки

- 4 KPI-плитки
- Производство (дни/бригады)
- Усушка по рейсам
- Склад / Цены / Продажи
- Chart.js + пресеты периода

Визуал KPI (из макета)

Целевые плитки, которые кормит backend

Плитка	Значение	Подпись под значением
Выработка	тонн произведено	количество смен за период
Вывезено	тонн уехало по данным весовой	количество рейсов
Продано	выручка в рублях	проданные тонны
Усушка	процент потери веса	потеря в кг и её оценка в рублях

ВЫРАБОТКА

120,3_т

16 смен за период

ВЫВЕЗЕНО

88,0_т

19 рейсов по весам

ПРОДАНО

3,12 млн ₹

172,8 т за период

УСУШКА

1,8 %

-1 590 кг ≈ 29 тыс ₹

Формулы показателей

Как Python считает KPI из сырых строк SQL

Выработка, т $\text{SUM}(\text{кол-во строк смен}) / 1000$

Вывезено, т $\text{SUM}(\text{вес весовой отгрузки}) / 1000$

Усушка, кг $\text{SUM}(\text{вес тюков} - \text{вес весовой})$

Усушка, % $(\text{тюки} - \text{весовая}) / \text{тюки} \times 100$

Усушка, Р $\text{усушка_т} \times \text{средняя цена корзины продаж}$

Индекс цены $\text{цена_месяца} / \text{цена_базы} \times 100$

Доля покупателя $\text{сумма контрагента} / \text{выручка периода}$

Итог

Что получилось на backend

Разведка

- MCP → структура SQL
- имена _Document/_Fld
- разово, не в рантайме

Доступ

- SQL на .10, не MCP
- pymssql + .env
- независимый контур

Отдача

- живые SELECT
- Flask /api/data
- KPI + 5 блоков UI

Документация: [dashboard/README.md](#) · контекст: [CLAUDE.md](#) · код: [app.py](#), [db.py](#)

MCP — структура; SQL — данные UI. Прототип на live SQL (без OData/витрины). Смены в копии — в основном май–июнь 2026.